

GPT-4o-mini có chọn mutant giỏi hơn chọn ngẫu nhiên không?

GPT-4o-mini-based Mutant Selection for Java Mutation Testing

Thành viên

Phạm Mạnh Quỳnh · Trần Trúc Vỹ · Phan Nguyễn Đình Khôi · Nguyễn Trường Vinh · Nguyễn Nhật Anh

FPT University, HCM — 2026

Mutation testing rất tốt — nhưng quá đắt để chạy thật

01

Sinh ra quá nhiều mutant

Chỉ 3 project cỡ nhỏ, 10 version mỗi project đã ra 11,040 mutant.

02

Mỗi mutant = chạy lại cả test suite

Chi phí nhân lên hàng nghìn lần.
Pipeline CI không có đủ thời gian để chạy hết.

03

Nên trong thực tế team bỏ luôn

Kỹ thuật tốt nhưng không ai dùng — vì không ai chờ nổi một lần chạy đầy đủ.

CÂU HỎI THẬT CỦA DEVELOPER

“Tôi chỉ đủ thời gian chạy 10% số mutant. Vậy phải chọn 10% NÀO?”

Đây chính là bài toán mutant selection — cùng ngân sách, chọn khéo hay chọn bừa thì khác nhau bao nhiêu?

LLM đã có mặt khắp mutation testing — trừ việc CHỌN

RESEARCH GAP

LLM trong mutation testing đã làm 3 việc:

- **SINH mutant** — Wang 2025, SMART 2026, LLMorpheus
- **LỌC equivalent mutant** — GEM-LLM (precision 98%)
- **SINH test từ mutant** — MuTAP, MUTGEN

Chưa ai dùng LLM để CHỌN mutant từ một pool có sẵn, rồi so trực tiếp với random cùng ngân sách.

Nghiên cứu gần nhất — Jimenez et al. (2018)

So n-gram naturalness với random trên 5 project Defects4J $\rightarrow A_{12} \approx 0.57$ (small): xếp hạng bằng language model KHÔNG hơn random một cách có ý nghĩa thực tế.

Câu hỏi bỏ ngỏ: LLM hiện đại có làm tốt hơn n-gram ở đúng vai trò đó không?

RQ1

Trên các project Java của Defects4J, GPT-4o-mini chọn mutant có đạt mutation score cao hơn random fixed-seed không, khi cả hai cùng chọn top 10%?

GIẢ THUYẾT — CHỐT TRƯỚC KHI CHẠY

H0: $MS_{GPT} \leq MS_{random}$

H1: $MS_{GPT} > MS_{random}$

Tiêu chí thành công (pre-registered)

$p < 0.05$ VÀ $r_{rb} \geq 0.30$ (medium)

Cùng một pool, cùng một ngân sách — chỉ khác cách chọn



Vì sao thiết kế này sạch: hai nhánh dùng CHUNG một pool mutant của PIT và CHUNG một ngân sách (1,119 mutant). Không nhánh nào sinh mutant mới, nên mọi chênh lệch mutation score đều quy về đúng một thứ: chất lượng của quyết định chọn.

Sự cố đã xử lý minh bạch: lần chạy PIT đầu tiên Codec ra 100% NO_COVERAGE (0/2,317 killed) vì thiếu JUnit 4 trên classpath. Sau khi sửa: 84.9% killed. Đã thêm auto-guard fail pipeline nếu project nào 0 killed hoặc >50% NO_COVERAGE.

Metric, kiểm định — và một tiêu chí đã phải sửa

METRIC

MS = Killed / Selected

Tỉ lệ mutant trong tập ĐÃ CHỌN bị test suite giết.

Tính ở hai đơn vị — cố ý

Per-group (37 group) — đơn vị pre-registered, dùng cho kiểm định ghép cặp.

Pooled mutant-weighted (1,119 mutant) — phản ánh cái người dùng thực sự nhận được.

Slide 6–7 sẽ cho thấy hai đơn vị này kể hai câu chuyện khác nhau — và bản thân điều đó là một phát hiện.

STATISTICAL TEST

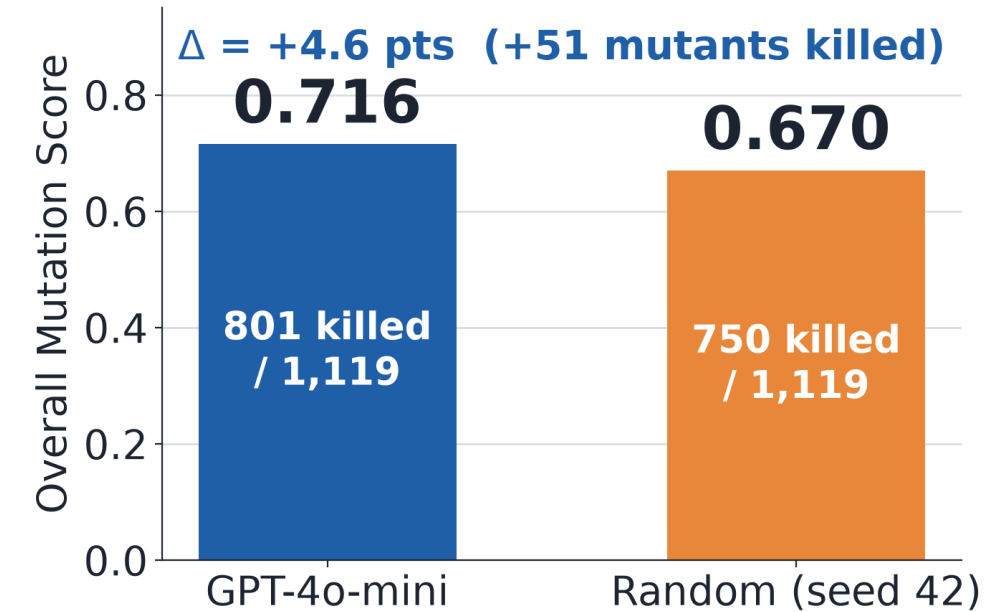
- **Wilcoxon signed-rank một phía** — ghép cặp theo (project, version, class), $\alpha = 0.05$. Dữ liệu ghép cặp theo thiết kế, MS không giả định phân phối chuẩn.
- **Effect size rank-biserial r_{rb}** — ngưỡng ≥ 0.30 (medium, Cohen).
- **Ngưỡng 0.30 không tùy tiện** — Jimenez đo $A_{12} \approx 0.57$ (small) và kết luận là chưa đủ giá trị thực tế.

AMENDMENT — ĐÃ BỎ TIÊU CHÍ MEDIAN

Tiêu chí gốc còn đòi $\text{median}(\Delta MS) > 0$. Nhưng số mutant chọn trong mỗi group k chạy từ 1 đến 173: group $k=1$ chỉ có $MS \in \{0, 1\}$, $k=2$ nhảy bước 0.5 → 11/37 cặp hòa tuyệt đối, median bị ghim = 0 bất kể hiệu ứng đi hướng nào.

Đây là lỗi đặc tả của tiêu chí → bỏ median, GIỮ NGUYÊN p và r_{rb} .

Tổng thể GPT thắng — nhưng test pre-registered không đạt



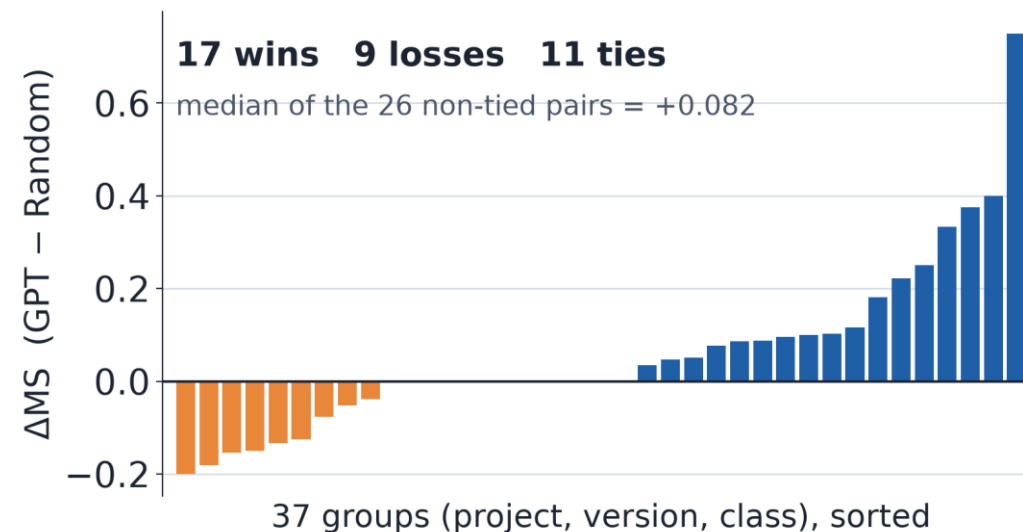
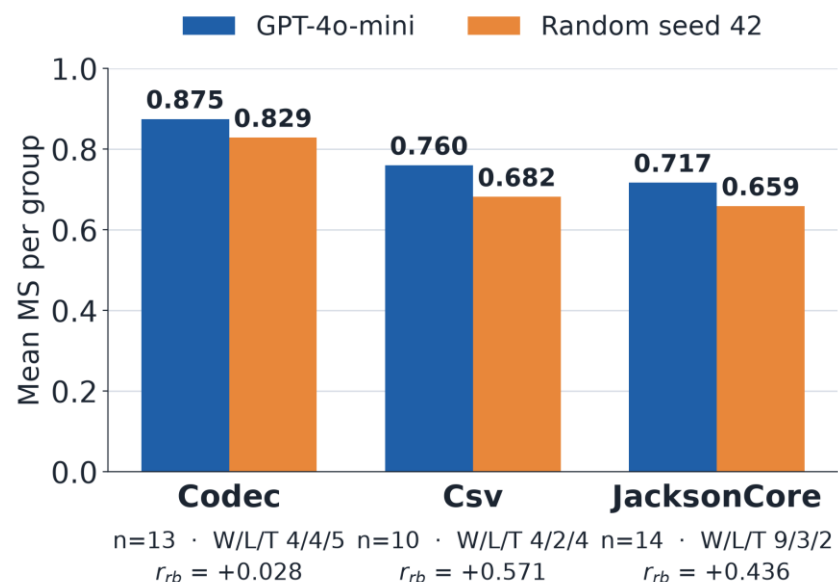
GHÉP CẶP THEO GROUP — WILCOXON MỘT PHÍA

Subset	n	W / L / T	p	r_rb
Pooled (tất cả)	37	17/9/11	0.0591	+0.350
Codec	13	4/4/5	0.4922	+0.028
Csv	10	4/2/4	0.1250	+0.571
JacksonCore	14	9/3/2	0.1018	+0.436

$p = 0.0591 > \alpha = 0.05$

→ KHÔNG bác bỏ được H_0 ở đơn vị pre-registered — dù $r_{rb} = +0.350$ đã đạt ngưỡng medium. Không subgroup nào đạt $p < 0.05$; Csv có effect lớn nhất (+0.571) nhưng chỉ $n = 10$.

Ở cấp độ mutant, lợi thế là thật — nhưng không đều



Mutant-weighted z-test

$z = 2.34$ $p = 0.0097$

bác bỏ H_0 ở cấp độ mutant

Monte Carlo 5,000 lượt random

$p = 0.0002$ $+3.64$ sd

GPT vượt 4,999/5,000 lượt; seed 42 nằm ở percentile 52 — một lượt bốc rất bình thường

Không phải do né NO_COVERAGE

0.864 vs 0.810

chỉ xét mutant có coverage, GPT vẫn hơn

Vì sao hai kết luận trái nhau — và điều chưa trả lời được

VÌ SAO?

- **Lợi thế tập trung, không đều.** 63% số mutant giết thêm (32/51) đến từ 3 group parser lớn nhất của JacksonCore — code nhiều nhánh điều kiện, đủ chỗ để LLM phân biệt mutant nông và mutant sâu.
- **Codec bị hiệu ứng trần.** Pool đã KILLED sẵn 84.9% → chọn khéo và chọn bù hời tụy về cùng kết quả (4 thắng / 4 thua / 5 hòa).
- **Đơn vị phân tích quyết định kết luận.** Test theo group cho group $k=1$ và group $k=173$ cùng trọng số, cộng 30% cặp hòa → triệt tiêu thông tin xếp hạng của Wilcoxon. Không mâu thuẫn — là hai câu hỏi khác nhau: “GPT thắng ở đa số class?” (chưa chứng minh được) và “tập GPT chọn giết nhiều mutant hơn?” (có).

HẠN CHẾ

- ▶ **N = 37 group → power thấp.** Bài học: số CLASS mới quyết định n, không phải số project.
- ▶ **Thang điểm 1–5 quá thô.** Sinh ra 11/37 cặp hòa — nguồn mất mát thông tin chính.
- ▶ **Chỉ một seed (42) cho random.** Đã bù bằng Monte Carlo 5,000 lượt; seed 42 ở percentile 52.
- ▶ **Phạm vi hẹp.** 3 project Java / 20 class / 1 mô hình — không tổng quát hoá cho mọi project, ngôn ngữ hay LLM khác.
- ▶ **MS ≠ bắt được bug thật.** “Usefulness” mà prompt hỏi không trùng hoàn toàn với “killability” mà MS đo.

TRẢ LỜI RQ1: PARTIAL SUPPORT

Kiểm định pre-registered (theo group): $p = 0.059$, $r_{rb} = +0.350$, $N = 37 \rightarrow$ chưa đủ bằng chứng bác bỏ H_0 .

Ở cấp độ mutant: 0.716 vs 0.670 (+4.6 điểm), z-test $p = 0.0097$, Monte Carlo $p = 0.0002 \rightarrow$ tín hiệu chọn là thật.

Nhưng không đều: mạnh ở parser phức tạp (JacksonCore 9 thắng / 3 thua), biến mất khi test suite đã gần bão hoà (Codec, MS nền 84.9%).

HƯỚNG PHÁT TRIỂN

01

Tăng power đúng đòn bẩy

Thêm CLASS trong cùng project (chạy PIT full scope thay vì chỉ class bị patch đung vào), không phải thêm project.

02

Kiểm định giả thuyết độ phức tạp

Phân tầng kết quả theo cyclomatic complexity để xác nhận trực tiếp mẫu "LLM giúp nhiều nhất ở code phức tạp".

03

Tăng độ phân giải điểm

Thay thang 1–5 bằng pairwise ranking hoặc kill probability đã hiệu chỉnh, để giảm số cặp hòa.

Replication package: github.com/KhoiPhanDinh/RT-SWT-003-nhom3

Cảm ơn thầy/cô đã lắng nghe.